

The SQL Masterclass: From Architecture to Extraction

A comprehensive guide to database constraints, manipulation, and advanced querying.



Syntax Color Key



COMMAND



CLAUSE



CONSTRAINT



VALUE

NOT NULL



NOT NULL: Columns cannot have a null value.

UNIQUE



UNIQUE: All values in the column are different.

DEFAULT



DEFAULT: Sets the fallback value of a column if none is provided.

```
salary INT DEFAULT 25000
```

CHECK



CHECK: Limits the specific values allowed in a column.

```
CONSTRAINT age_check CHECK  
(age >= 18 AND city= "Delhi")
```



The Unique Identifier (PRIMARY KEY)

- ◆ Uniquely identifies each row.
- ◆ Must be NOT NULL.
- ◆ Strictly only 1 Primary Key per table.

```
CREATE TABLE temp (  
  id int not null,  
  PRIMARY KEY (id)  
);
```

The Relationship Builder (FOREIGN KEY)

- ◆ Refers to the primary key in another table.
- ◆ Prevents actions that would destroy links.
- ◆ Can have duplicate and null values.
- ◆ Multiple Foreign Keys are permitted.

```
CREATE TABLE temp (  
  cust_id int,  
  FOREIGN KEY (cust_id) REFERENCES customer(id)  
);
```

Data Manipulation Language (DML)

Add



```
INSERT INTO table_name (colname1, colname2)
VALUES (col1_v1, col2_v1);
```

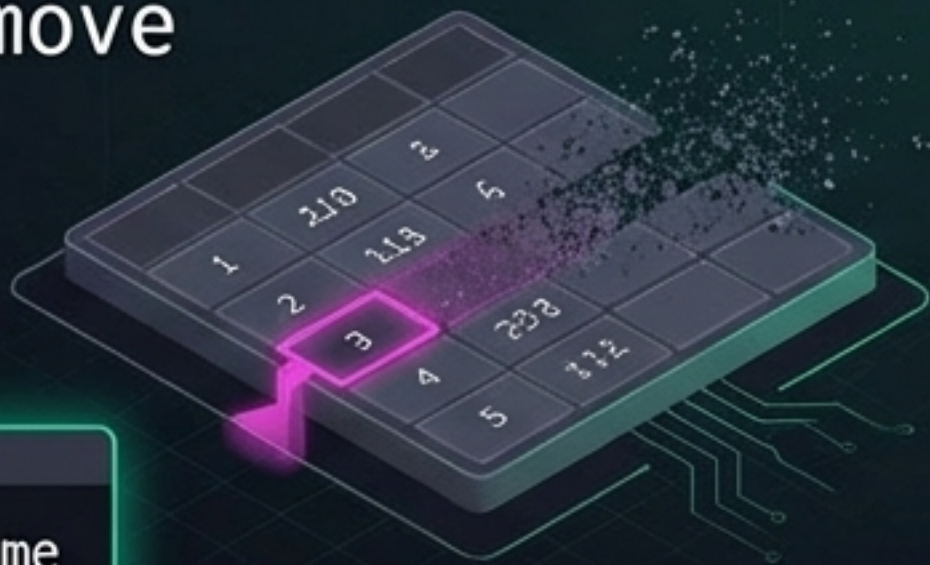
Modify



```
UPDATE table_name
SET col1 = val1
WHERE condition;
```

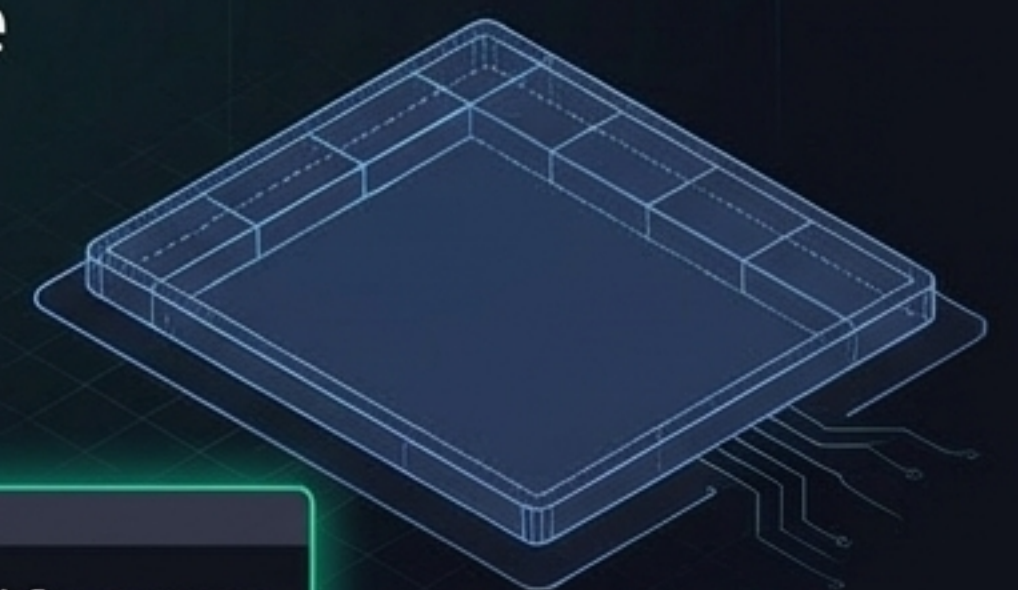
```
UPDATE table_name
SET col1 = val1
WHERE condition;
```

Precision Remove



```
DELETE FROM table_name
WHERE condition;
```

Bulk Remove



```
TRUNCATE TABLE table_name;
```

Schema Modification (DDL)

Expand Schema



```
ALTER TABLE table_name  
ADD COLUMN column_name  
datatype constraint;
```

Contract Schema



```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Rebrand Schema



```
ALTER TABLE table_name  
RENAME TO new_table_name;
```

The Core Engine: SELECT & LIMIT

Select Specific Columns



```
SELECT col1, col2  
FROM table_name;
```

Select All Data



```
SELECT *  
FROM table_name;
```



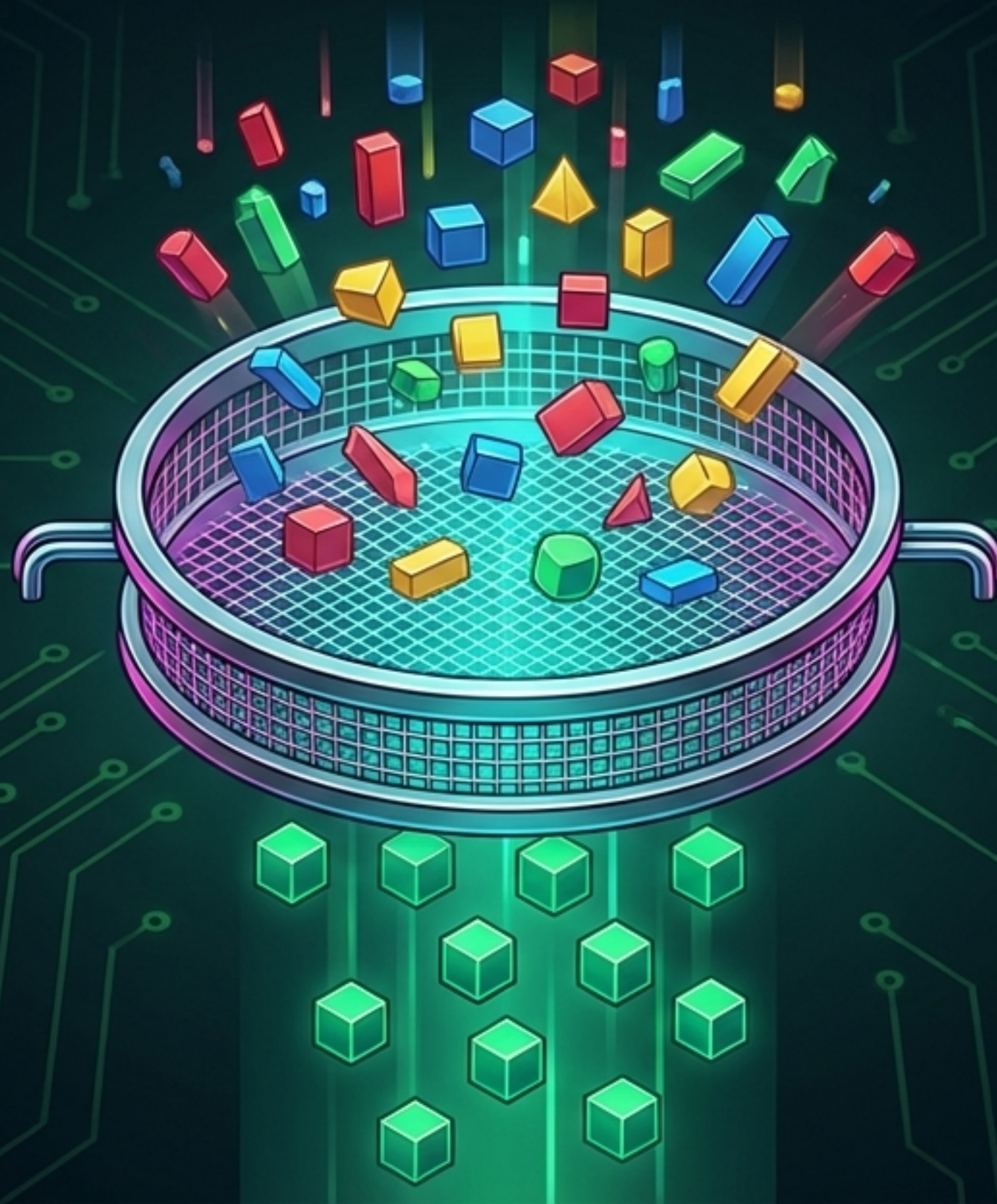
The Output Valve

Sets an upper limit on the number of rows returned.

```
LIMIT number;
```

Precision Extraction: The WHERE Clause

Define strict conditions to isolate the exact rows you need from the database while filtering out the noise.



```
SELECT  
    col1, col2  
FROM  
    table_name  
WHERE  
    conditions;
```

The Operator Toolkit



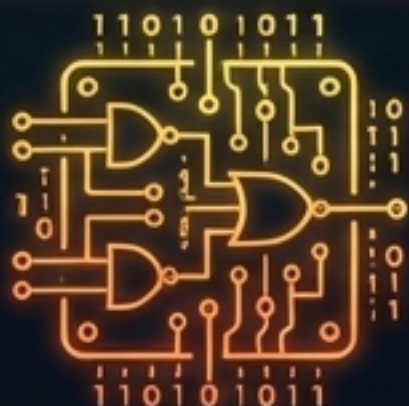
Arithmetic Operators

+ (addition), - (subtraction), * (multiplication),
/ (division), % (modulus)



Comparison Operators

= (equal to), != (not equal to), > (greater than),
>= (greater than or equal to), < (less than),
<= (less than or equal to)



Bitwise Operators

& (Bitwise AND), | (Bitwise OR)

Logical Operators



AND (checks for both conditions to be true)



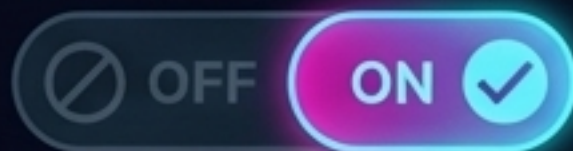
OR (checks for one of the conditions to be true)



BETWEEN (selects for a given range)



IN (matches any value in the list)

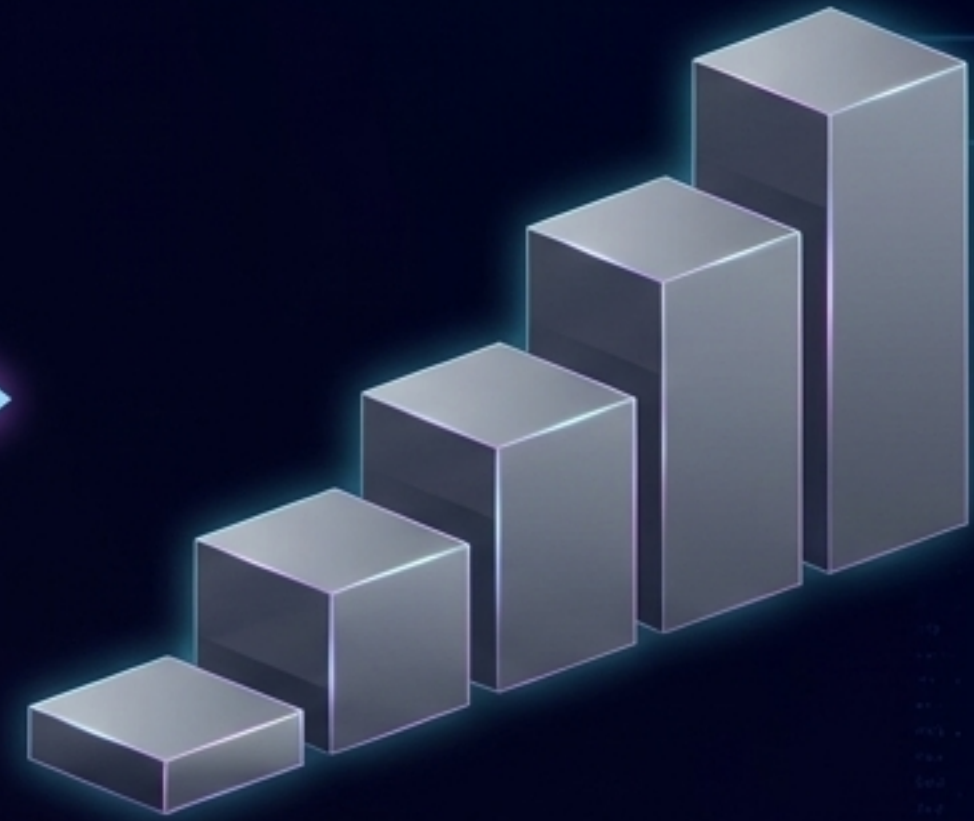


NOT (to negate the given condition)

Sorting the Output: ORDER BY



ORDER BY

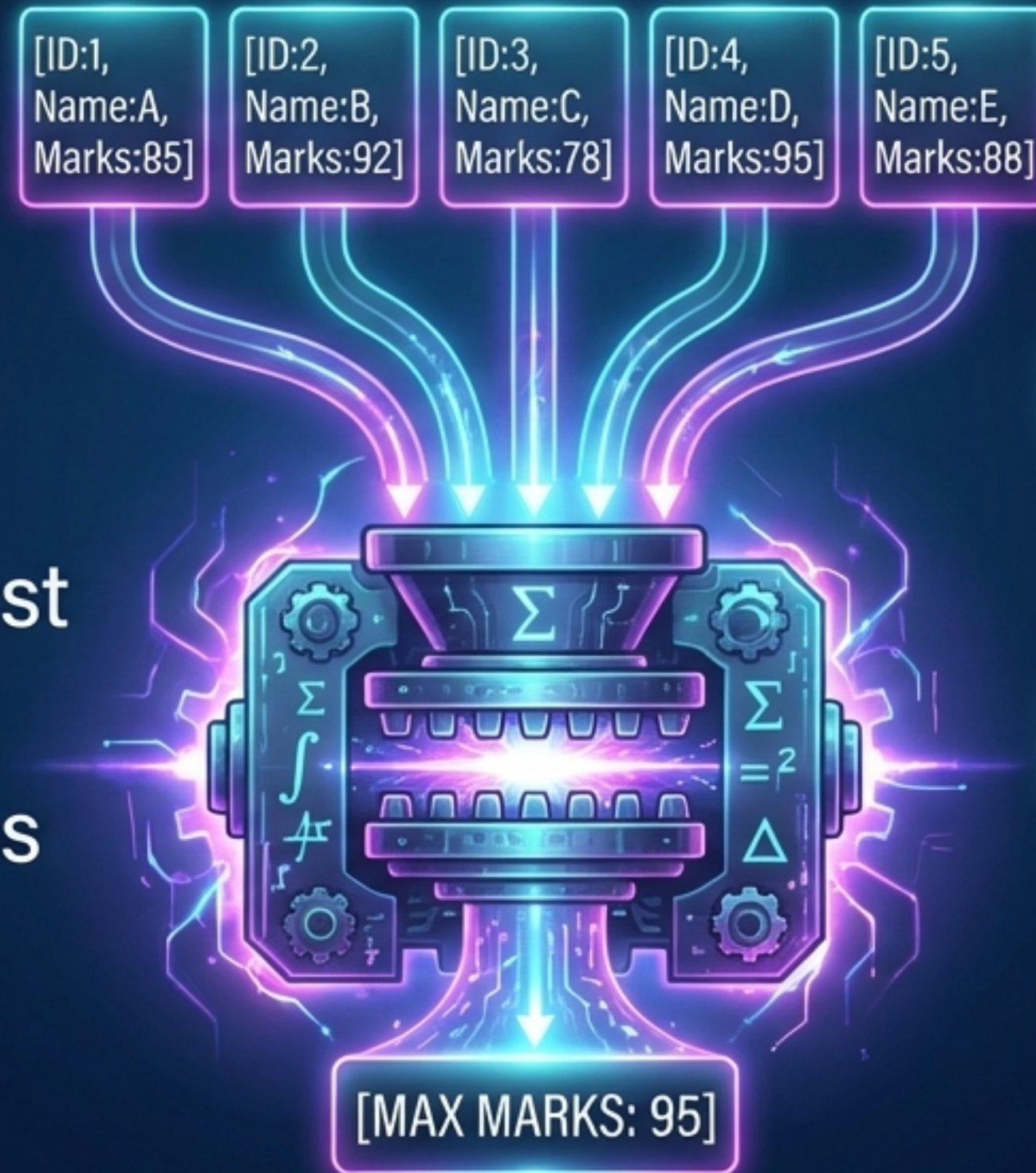


Organize the final result set in ascending (ASC) or descending (DESC) sequence.

```
SELECT col1, col2  
FROM table_name  
ORDER BY col_name(s) ASC;
```

Mathematical Summaries: Aggregate Functions

- **COUNT()**: Totals the number of rows.
- **MAX()**: Finds the highest value.
- **MIN()**: Finds the lowest value.
- **SUM()**: Adds all values together.
- **AVG()**: Calculates the mean.



```
SELECT max  
      (marks)\nFROM student;
```

Structuring Data: The GROUP BY Clause



```
SELECT col1, col2  
FROM table_name  
GROUP BY col_name(s);
```

Collects data from multiple records and groups rows that have the same values into distinct summary rows. Generally used alongside an aggregation function.

Diagnostic Matrix: WHERE vs. HAVING

WHERE (Pre-Aggregation)

WHERE is strictly for the **TABLE**.

- - Filters raw individual rows **BEFORE** any mathematical grouping occurs.

HAVING (Post-Aggregation)

HAVING is strictly for the **GROUP**.

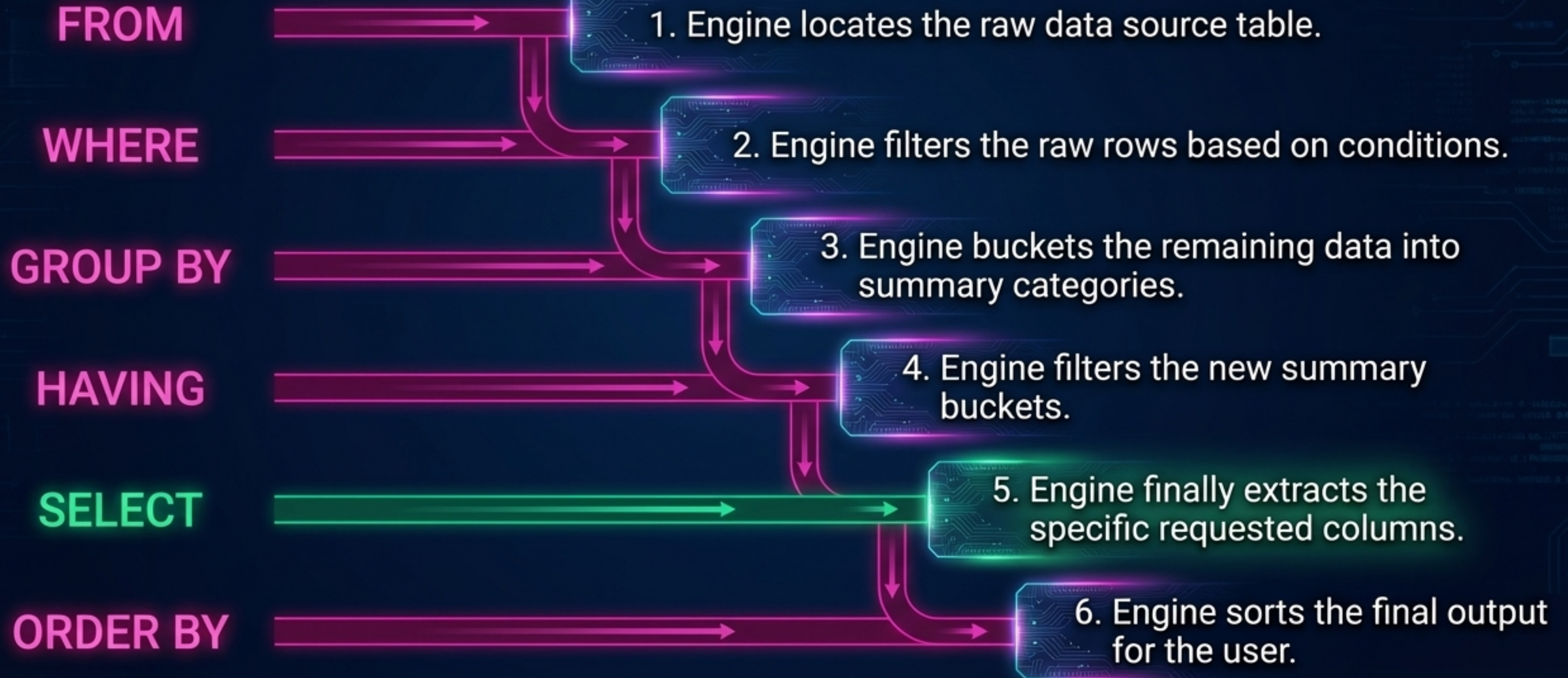
- - Applies conditions only **AFTER** grouping has occurred.
- - Grouping is absolutely necessary for **HAVING** to function.

```
SELECT col1, col2\nFROM table_name\nGROUP BY col_name(s)\nHAVING condition;
```

Synthesis: The Master Execution Order

Written Syntax

Chronological Engine Execution



The Complete Architecture

```
1 SELECT department, COUNT(id), AVG(salary)
2 FROM employees
3 WHERE status = "Active" ← Filters raw rows first
4 GROUP BY department ← Buckets the remaining data
5 HAVING AVG(salary) >= 50000 ← Filters the new buckets
6 ORDER BY COUNT(id) DESC
7 LIMIT 10;
```